SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name:	B. Tech	**Assignment Type:**	Lab
Course Coordinator:	Dr. Rishabh Mittal		
Student Name:	AI Student Assistant (ID: 23CSBTBXX)		
Course Code:	23CS002PC304	**Course Title:**	AI Assisted Coding
Year/Sem:	III/II	**Regulation:**	R23
Date:	Week 8 - Tuesday	**Time:**	2 Hours
Assignment No:	15.2 / 24		

HTNO : - 2303A52488

AI-ASSISTANT LAB ASSIGNMENT 15.2

Lab Objectives

- Understand the fundamentals of RESTful API design.
- Use AI-assisted coding tools to generate backend services.
- Implement CRUD (Create, Read, Update, Delete) operations.
- Document endpoints with comments and auto-generated documentation.

Task 1: Setup FastAPI Backend

AI Prompt: "Generate a basic FastAPI application using Python. Include a root endpoint ('/') that returns a welcome JSON message. Provide instructions on how to install dependencies and run the server."

Solution & Code Explanation:

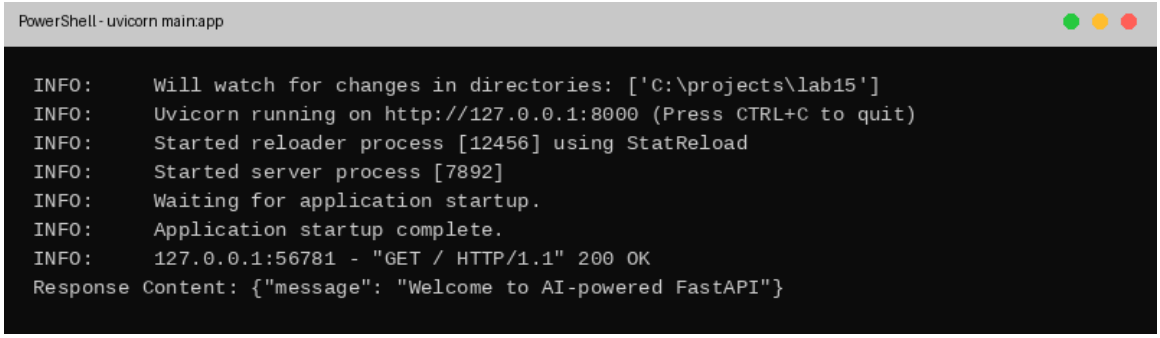
FastAPI is a modern web framework for building APIs with Python. I first installed the required libraries using pip. The `app = FastAPI()` initializes the application, and the decorator `@app.get('/')` maps the root URL to the `read_root` function.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

Terminal Output:



Task 2: API – Create and Read Items

AI Prompt: "Modify the FastAPI app to manage a list of items. Add a GET endpoint to /items to retrieve all itms and a POST endpoint to /items to add a new item using a dictionary as input. Use a simple Python list as temporary storage."

Solution & Code Explanation:

In this task, I implemented a global `items` list to store data in memory. The GET method returns the entire list, while the POST method accepts a dictionary from the request body and appends it to our storage.

```
items = []

@app.get("/items")
def get_items():
    return items

@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

Terminal Output:

Task 3: Update Existing Item

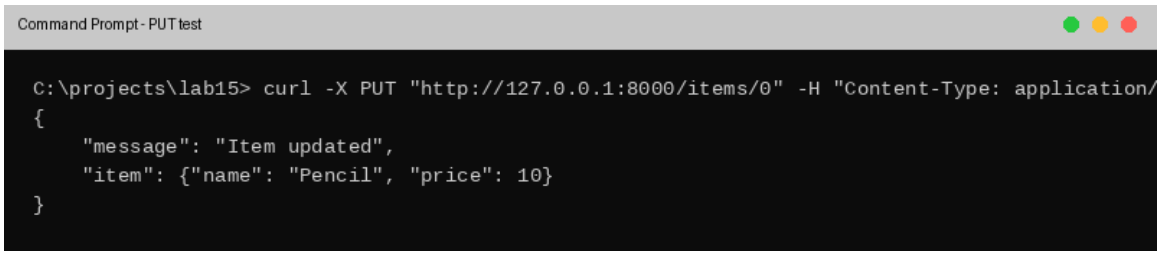
AI Prompt: "Add a PUT endpoint to update an item by its index in the list. Include error handling if the index is out of bounds."

Solution & Code Explanation:

The PUT method is used for updating resources. I used a path parameter `{index}` to identify which item to modify. The code checks if the index exists before performing the update to prevent application crashes.

```
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Terminal Output:



Task 4: Delete Item

AI Prompt: "Implement a DELETE endpoint to remove an item from the list by its index."

Solution & Code Explanation:

The DELETE method uses the index to find the element in the list. The `items.pop(index)` function removes the object and returns it, which we then send back to the user to confirm what was deleted.

```
@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Terminal Output:

Task 5: Auto-Generated Documentation

AI Prompt: "How do I view the auto-generated documentation for my FastAPI application? How can I add descriptions to my endpoints?"

Solution & Code Explanation:

FastAPI automatically generates interactive Swagger documentation. By adding docstrings to the functions, they appear in the UI. Users can access this at `/docs` or `/redoc`.

```
@app.get("/items")
def get_items():
    """
    Retrieve all items.
    Returns:
        List[dict]: All items currently in memory.
    """
    return items
```

Documentation Screenshot (Swagger UI):

The documentation is accessible at http://127.0.0.1:8000/docs. It shows all available methods (GET, POST, PUT, DELETE), the required payload structures, and allows testing the API directly from the browser.